DEEP LEARNING ON POINT CLOUDS

PointNet++

Deep Hierarchical Feature Learning on Point Sets in a Metric Space

A visual walkthrough of the architecture, modules, and intuition

Based on the paper by Qi, Yi, Su & Guibas — Stanford University

MOTIVATION

Why PointNet++ Was Needed

PointNet

"Understand the whole object directly."

- Processes ALL points → one global feature.
- Treats every point independently — uses a shared MLP and a single global max-pool.
- Captures "what" the object is, but loses how its small parts are arranged locally.
- Struggles with fine local structure, dense scenes, and segmentation tasks.

PointNet++

"Understand local patterns first, then build up."

- Learns features hierarchically — small neighborhoods first, then larger ones.
- Captures local geometric structure: edges, curves, parts.
- Works much better for segmentation, complex scenes, LiDAR, dense 3D understanding.
- Mirrors how CNNs learn: edges → textures → parts → objects.

INTUITION

The Big Idea: Hierarchical Learning

Real 3D objects are hierarchical. A chair is built from parts, and parts are built from tiny geometric patterns.

1

LEVEL 1

Tiny local patterns

- small curves
- edges
- nearby shapes

2

LEVEL 2

Object parts

- chair legs
- seat corners
- armrest shapes

3

LEVEL 3

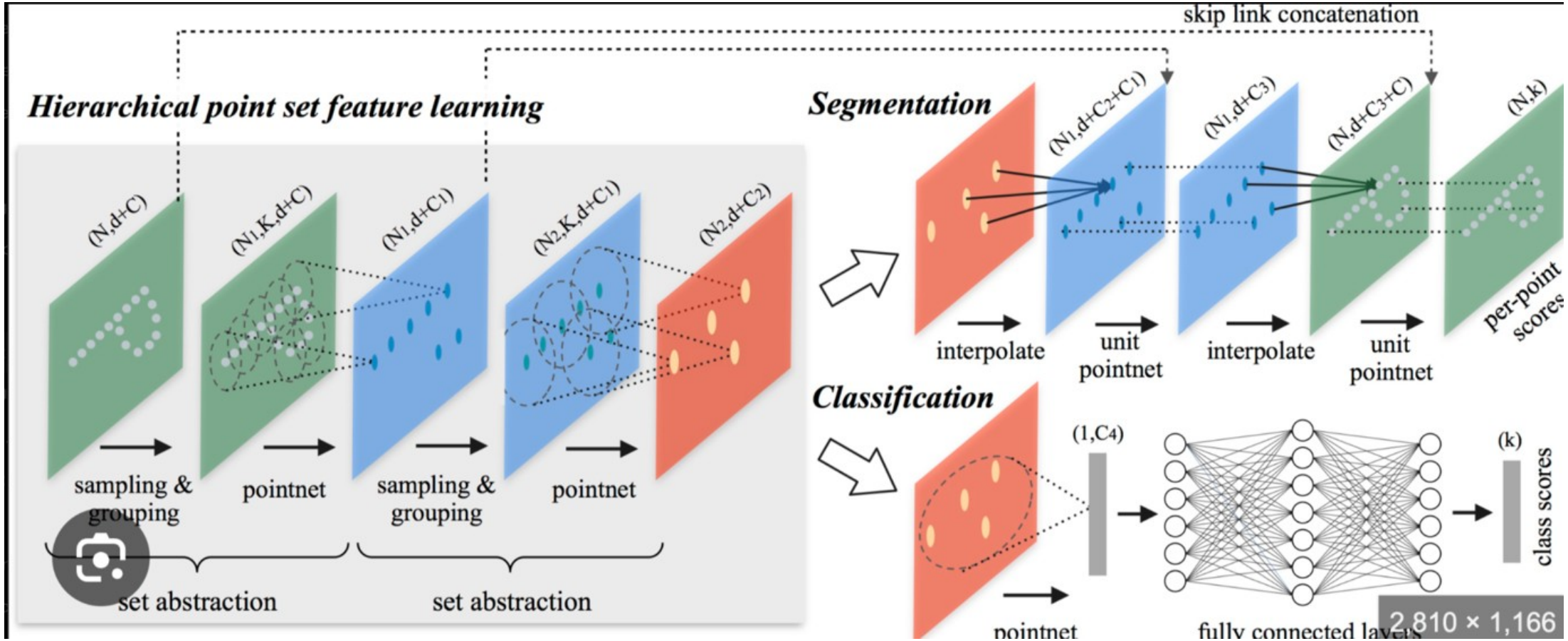
Full object

- the entire chair
- global geometry
- what it is

ARCHITECTURE

Overall Pipeline of PointNet++

The full architecture has two halves: an Encoder of Set Abstraction (SA) layers and, for segmentation, a Decoder of Feature Propagation (FP) layers with skip connections.



Original architecture diagram from the PointNet++ paper

CORE BUILDING BLOCK

The Set Abstraction (SA) Layer

The SA layer is the repeated block at the heart of PointNet++. It does three things — and is then stacked over and over.

1

Sampling

Pick centroids using Farthest Point Sampling (FPS).

2

Grouping

For each centroid, find nearby points via Ball Query.

3

Mini-PointNet

Apply shared MLP + max-pool to each local region.

Mental model: *after each SA layer, a "point" gradually becomes a representative of a local region — not just a single (x, y, z).*

Step 1 — Sampling Layer (FPS)

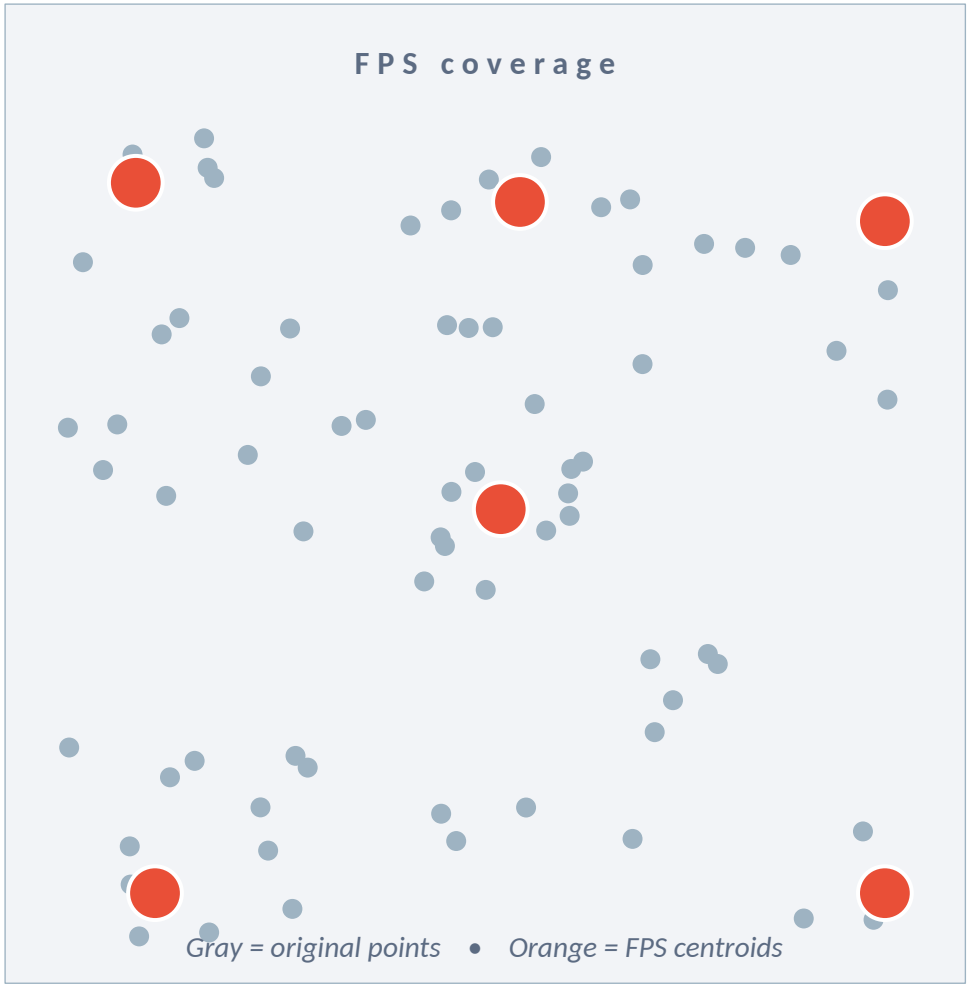
Why sampling?

- Computing neighborhoods around every point in a 100k-point cloud is prohibitively expensive.
- Instead, PointNet++ picks a small set of representative points — called centroids.
- These centroids become the centers of local neighborhoods in the next step.

Farthest Point Sampling (FPS)

1. Pick one point at random.
2. Pick the point farthest from already-chosen points.
3. Repeat until you have K centroids.

- Result: centroids spread evenly over the object — better coverage than random sampling.



INSIDE AN SA LAYER

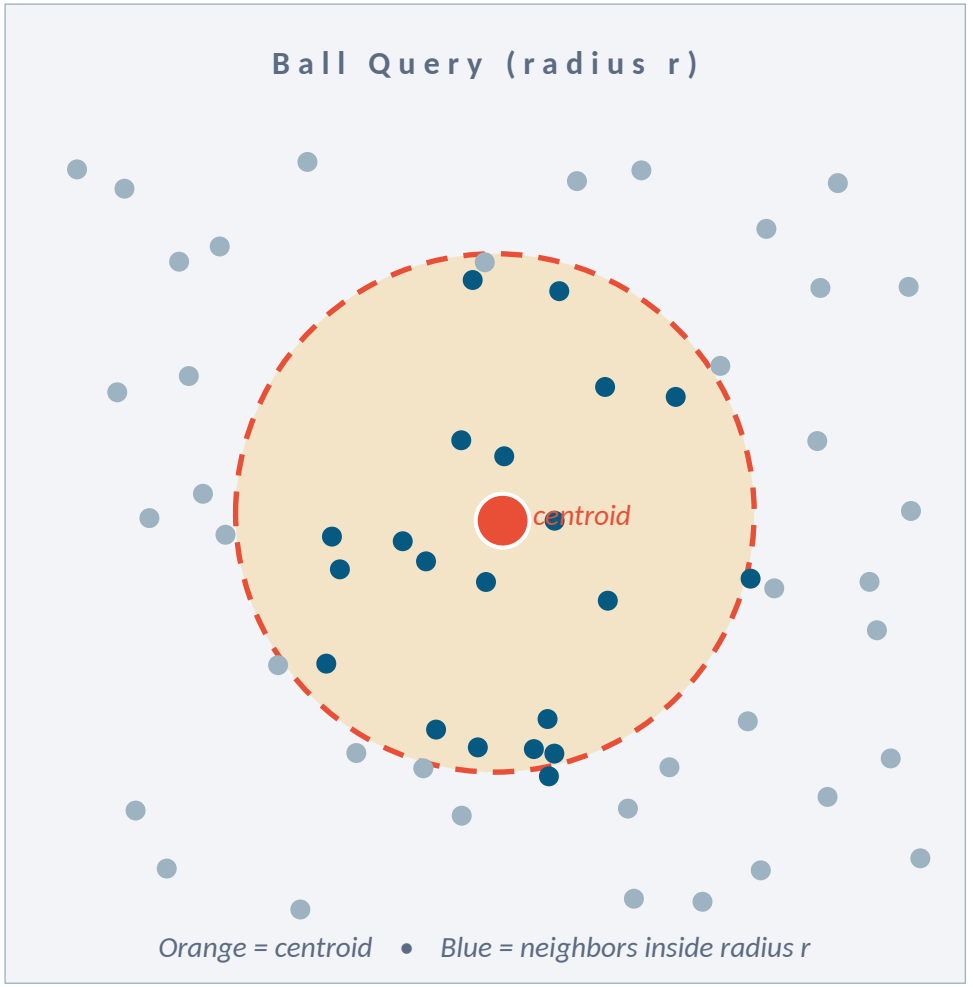
Step 2 — Grouping Layer (Ball Query)

What grouping does

- For each centroid, collect nearby points to form a local region (e.g., 32 neighbors).
- Each region is later summarized by a tiny PointNet.

Ball Query vs. kNN

- Ball Query: "all points within radius r of the centroid."
- kNN: "the k nearest points" — but k stays fixed regardless of density.
- Ball Query preserves a fixed spatial scale, which matters geometrically.
- More robust when point density varies across the object.



INSIDE AN SA LAYER

Steps 3 & 4 — Local Coordinates + Mini-PointNet

Step 3 — Local Coordinates

Neighborhood points are re-expressed relative to the centroid.

- Replace global (x, y, z) with (x − xc, y − yc, z − zc).
- Local shape matters more than absolute position.
- A chair leg is a chair leg whether it sits on the left, right, or center of the scene.
- Gives the next module translation-invariant local geometry to work with.

Step 4 — Mini-PointNet

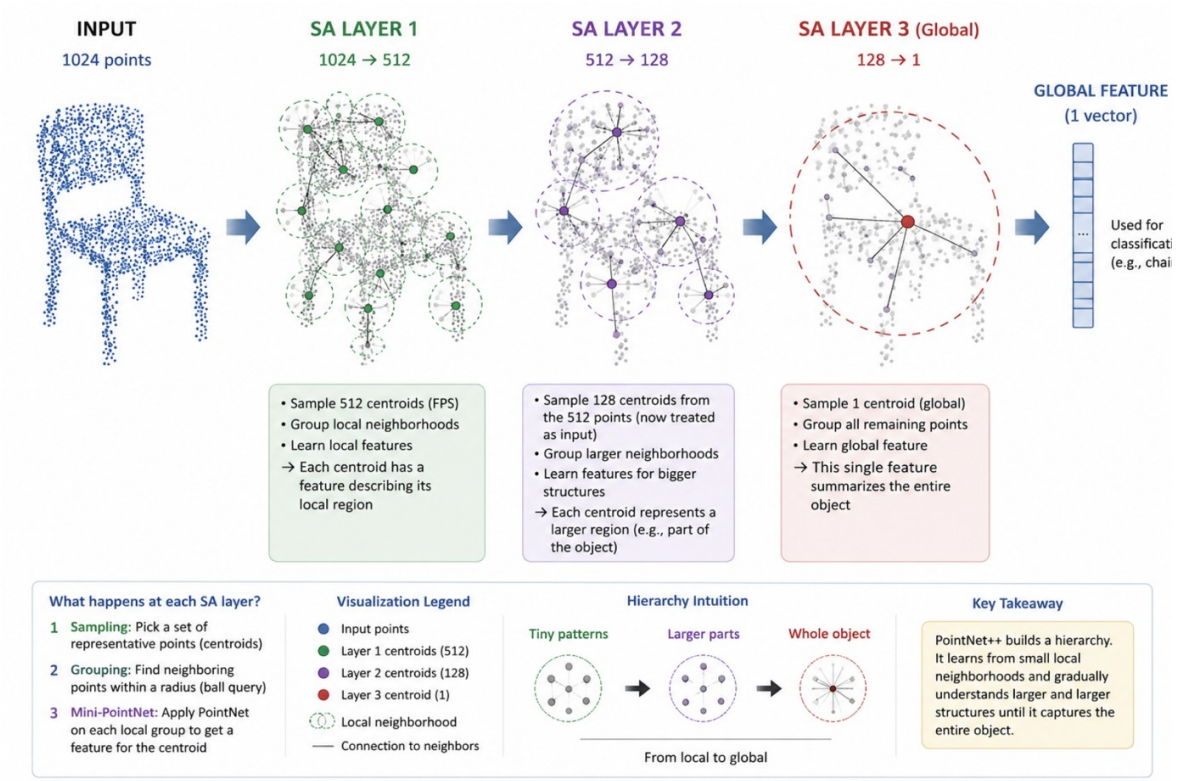
Run a tiny PointNet inside each local neighborhood.

- Each region (≈ 32 points) goes through a shared MLP.
- A max-pool over the region produces one feature vector (e.g., 128-D).
- That vector summarizes the local geometry around the centroid.
- Effectively: PointNet inside neighborhoods — learns corners, curves, surfaces, edges.

PUTTING SA LAYERS TOGETHER

Hierarchical Point Set Feature Learning

Stacking SA layers gradually compresses 1024 points → coarser, more semantic representations, until a single global feature describes the whole object.



Chair example: each SA layer reduces points while building richer local features (1024 → 512 → 128 → 1 global vector).

HIERARCHY IN ACTION

What the Network Learns at Each Layer

SA Layer 1

Tiny neighborhoods

Learns

edges, small curves, local surface patches

SA Layer 2

Medium regions

Learns

chair legs, armrest shapes, seat corners

SA Layer 3

Whole object

Learns

full chair structure → 1 global feature vector

PART 2

Classification Head

From a global feature vector to a single class prediction.

From Global Feature to Class Scores

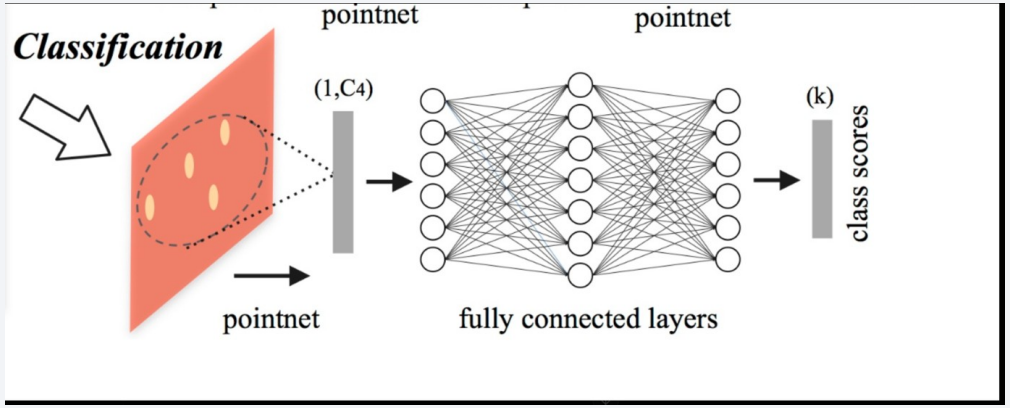
What's the (1, C) vector?

- One feature vector with C dimensions for the whole object.
- Compressed knowledge of the entire point cloud — not raw points anymore.
- Encodes learned patterns: "legs detected", "flat surface", "vertical structure", "symmetry"...
- Not human-readable — patterns are learned during training.

Fully Connected (FC) Layers

- Standard MLP: e.g., 1024 → 512 → 256 → k classes.
- Gradually convert geometry understanding into class probabilities.
- Final softmax produces a probability per class.
- No FPS, ball query, or grouping here — geometry is already understood by SA layers.

Classification head



Global feature → FC layers → class scores (k)

CLASSIFICATION

Final Output: Scores → Softmax → Class

Raw scores (logits)

Class	Score
Chair	9.2
Table	1.1
Lamp	−0.5
Car	−3.0
Airplane	−5.2

After softmax (probabilities)

Class	Probability
Chair	96%
Table	3%
Lamp	1%
Car	~0%
Airplane	~0%

PREDICTION

Chair

Highest softmax probability wins → final class label.

END TO END

Complete Classification Flow

SA layers (geometry)

FC layers (decision making)

Final prediction = arg max of class scores

PART 3

Segmentation Head

Predict a label for every point — not just the whole object.

WHAT'S DIFFERENT?

Segmentation vs. Classification

Classification

"What object is this?"

- One prediction for the entire point cloud.
- Uses only the global feature vector.
- Example output: Chair

Segmentation

"What is EACH point?"

- One prediction per point.
- Needs both global meaning AND fine local detail.
- Example output: leg / seat / backrest / armrest …

WHY SEGMENTATION IS HARD

The Problem: We Lost Point-Level Detail

Encoder compresses points → deep understanding, but original point detail is lost.

Analogy

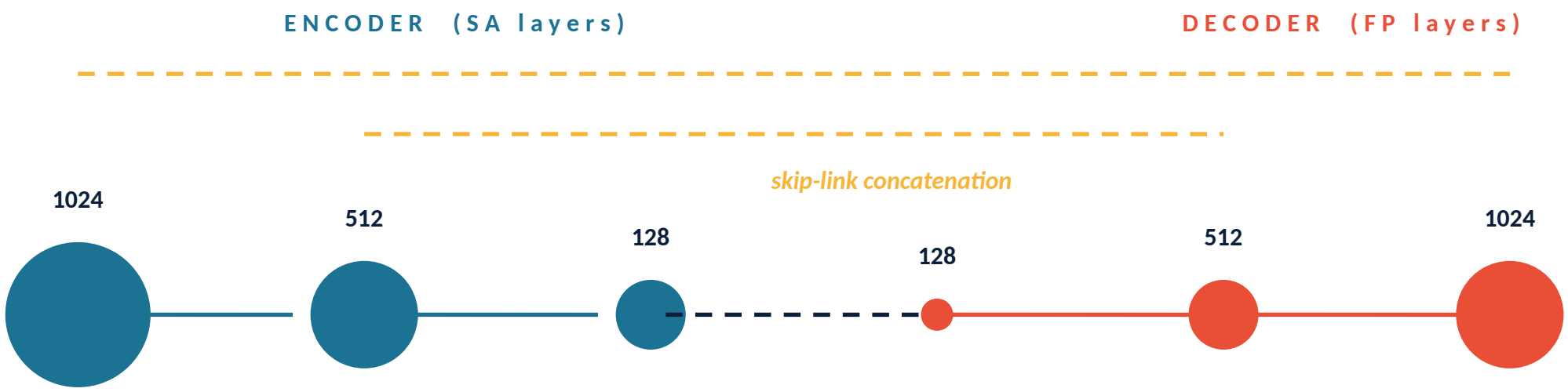
Seeing a city from airplane height

- You can tell it's Pune — that's the "object class".
- But you can't pick out exact streets or houses — that detail was compressed away.
- Segmentation needs both: global semantics AND precise local detail.

SEGMENTATION

The Decoder: Feature Propagation (FP) Layers

The encoder shrinks the point set. The decoder grows it back — propagating features from coarse points to all the original points.



Each FP step does: interpolate features → concatenate with skip-link → unit-PointNet refinement.

DECODER DETAIL

FP Step — Interpolation

PointNet++ spreads coarse-but-rich features back to finer point sets using inverse-distance-weighted interpolation.

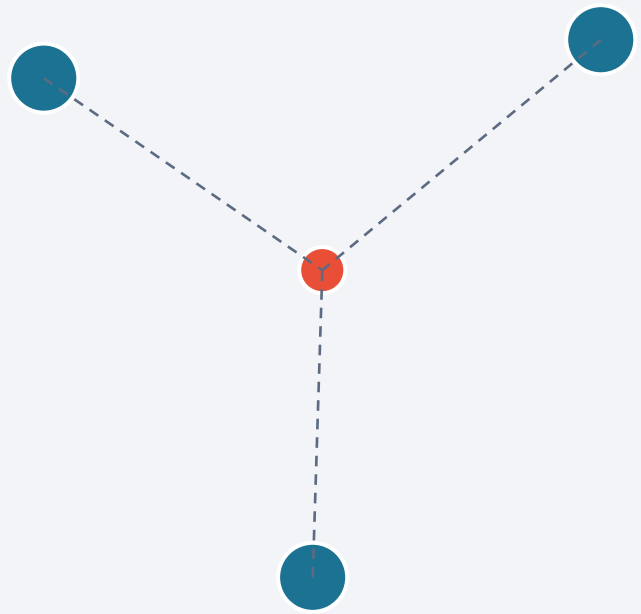
How it works

- For each fine point, find its k nearest coarse points.
- Estimate its feature as a weighted average of those coarse features.
- Closer coarse points get higher weight (weight ∝ 1 / distance).
- Interpolation does NOT create new understanding — it spreads learned semantics back to finer resolution.

Analogy

Estimate the temperature in a village from temperatures in Mumbai, Nashik, and Pune.

Inverse-distance weighted



Teal = coarse features • Red = fine point being filled in

THE GENIUS PART

FP Step — Skip Connections

Deep layers know WHAT. Shallow layers know WHERE EXACTLY. Skip-link concatenation gives us both.

Deep features (later SA layers)

- High semantics — know object parts.
- Low spatial detail — points were aggregated away.

Early features (earlier SA layers)

- High spatial detail — exact boundaries.
- Low semantics — local geometry only.

interpolated deep features + early detailed features → unit-PointNet refinement

…produces detailed, semantically meaningful per-point features — exactly what segmentation needs.

END OF SEGMENTATION PIPELINE

Final Per-Point Classifier

After repeating FP layers (128 → 512 → 1024), every original point has a rich feature with local detail + global context. A small MLP then predicts a class for each point.

Example output

Point	Predicted label
p1	chair leg
p2	seat
p3	backrest
p4	armrest
p5	chair leg
…	…

Per-point feature contains

- Local detail (from early-layer skip connections).
- Global context (from deep-layer features).
- Object-part awareness (from hierarchical SA).
- Smoothed predictions at boundaries (from unit-PointNet refinement).

RECAP

Key Takeaways

1 Hierarchical learning

Build up understanding from local patterns → parts → whole object — like CNNs for images.

2 Set Abstraction (SA) = Sample + Group + Mini-PointNet

FPS picks centroids; Ball Query forms regions; a tiny PointNet summarizes each region.

3 Local coordinates

Express neighborhood points relative to the centroid so geometry is translation-invariant.

4 Classification = global feature → FC → softmax

After SA layers compress to one vector, standard FC layers decide the class.

5 Segmentation needs both halves

Encoder (SA) understands; Decoder (FP) interpolates + uses skip connections to recover per-point detail.

6 Why PointNet++ matters

Works much better than PointNet for segmentation, complex scenes, LiDAR, and dense 3D understanding.

CREDITS

Original Research Paper

PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space

Charles R. Qi • Li (Eric) Yi • Hao Su • Leonidas J. Guibas

Stanford University

Conference

Advances in Neural Information Processing Systems (NeurIPS), 2017

Project page: stanford.edu — Qi et al., PointNet++

Slides prepared as a visual walkthrough for educational use.